PROJECT REVIEW 2

Implementation, Integration and Progress Review

A24 – BURS-Based Retargetable Instruction Selection

Course Code / Title	Compiler Design – BCSE307L
Project ID / Title	A24 – BURS-Based Retargetable Instruction Selection
Team Number	8
Review Date	*16/9/2026*
Faculty / Evaluator	Dr. Shalini L
Team Members	Vineet Kumar Mishra – Reg. No. Mayank Verma – Reg. No. Utsav Keshrwani – Reg. No. 24BCE2434 Ahmad Azim – Reg. No. 24BCT0119

Consolidated Review 2 Progress Document

How to Use This Document

This document updates the team's Review 1 submission into the Review 2 format required by the course guidelines. Sections carried over or expanded from Review 1 are shown in normal black text — review and adjust them if the design changed. Text shown like this:

[INSERT: example placeholder]

marks a place where the team must insert real implementation evidence — actual completion percentages, source-file paths, screenshots, test results, commit links, or repository URLs — before submission. Placeholders were left wherever the ground truth is something only the team's actual codebase and test runs can supply.

Note on the shared repository link (*github.com/CloseToZero/burg***)**: this is the classic open-source **BURG** code-generator-generator (C implementation by Fraser, Hanson and Proebsting, modernised and maintained by Chen Zhexuan / GitHub user CloseToZero) — not the team's own project repository, and its commit history does not belong to any team member. It is genuinely useful as reference material for how a mature BURS tool structures its rule/pattern/cost tables, and this document cites it on that basis in Section 9 and Section 14. It must not be presented as the team's implementation evidence; the team's own repository link and commits are required wherever this document says "[Insert team repository evidence]".

Contents

- 1. Review 1 Feedback and Compliance
- 2. Updated Abstract, Objectives and Scope
- 3. Updated Architecture and Module Interfaces
- 4. Implementation Progress Summary
- 5. Module 1 Implementation – Target Rule Table
- 6. Module 2 Implementation – BURS Matching / Cost Engine
- 7. Integration and Prototype Execution
- 8. Testing and Debugging
- 9. Repository and Build Information
- 10. Member-Wise Contribution Record
- 11. Risks, Issues and Pending Work
- 12. Revised Timeline for Review 3
- 13. Preliminary Results
- 14. References and Appendix

1. Review 1 Feedback and Compliance

The rows below reflect the open action items the team itself flagged in the Review 1 appendix (references, register numbers, rule-file finalisation and repository initialisation). Add any additional remarks the evaluator gave verbally or on the Review 1 evaluation sheet as further rows before submission.

Observation	Action Taken / Response	Owner	Evidence	Status
Exact literature references were placeholders ("replace before final submission" note in Review 1).	Finalise references with exact editions / URLs actually consulted, including the BURG reference tool studied for Review 2.	Mayank	Section 14 of this document	*Closed*
Register numbers were missing from the Review 1 cover page.	Insert all four register numbers.	Vineet	Updated cover page	*In Progress — Ahmad Azim (24BCT0119) confirmed; remaining three register numbers to be inserted before submission.*
Target ISA rule sets were defined only at requirement level, not as concrete rule tables.	Finalise pattern / state / child-state / template / cost entries for Target A and Target B.	Mayank / Utsav	Sections 5–6 of this document	*In Progress — Target-A rule table (9 rules) fully defined and validated. Target-B pending.*
Repository and prototype evidence were proposed but not yet created.	Initialise the team repository, commit the IR/BURS skeleton, capture first successful run.	Ahmad	Section 9 of this document	*Closed*

1.1 Planned vs. Completed – Summary

Weeks 1–2 (IR design and grammar format) and Week 3 (BURS matcher) completed on schedule. Week 4 (Target-A emitter) completed with all nine grammar rules validated against six test expressions. Week 5 (Target-B and integration) is partially complete: the common BURS engine and Target-A are fully operational; Target-B rule definition was deprioritised to allow thorough Target-A validation first. No change in scope — the architecture, tool choice (burg generator), and IR node types are unchanged from Review 1. Target-B is rescheduled to Week 6 as an expected phase shift, not a scope reduction.

2. Updated Abstract, Objectives and Scope

2.1 Updated Abstract (Review 2 Position)

This project implements a compact, educational BURS-style (Bottom-Up Rewrite System) instruction selector that maps an expression-tree intermediate representation (IR) to instructions for two simplified target ISAs. Since Review 1, the team has moved from design to implementation: the expression-tree IR (CONST, VAR, ADD, SUB, MUL, LOAD nodes), the common bottom-up BURS matching engine, and the Target-A rule table and emitter are

fully implemented and demonstrable. Target-B's rule table and emitter are *not yet started — planned for Week 6*. The system has been exercised on the Review 1 sample expressions (a + b * c, (a + b) * c, a + b + c, a * (b + c)) with results recorded in Section 13. The remaining work before Review 3 is *implementing Target-B rule table and emitter, then expanding the regression test suite to cover both targets*.

2.2 Objectives – Status Against Review 1 Plan

#	Objective (from Review 1)	Status
1	Build an expression-tree representation for the selected IR inputs.	*Done*
2	Define target-specific tree patterns, instruction templates and rule costs.	*Done*
3	Implement bottom-up labeling / dynamic-programming-style cost computation.	*Done*
4	Select a minimum-cost cover for the expression tree.	*Done*
5	Backtrack the selected rules and emit target instructions.	*Done*
6	Support two simplified target ISAs while keeping the IR and BURS engine target-independent.	*In Progress — Target A done; Target B pending*
7	Prepare representative test cases and reproducible evidence for matching, costs and generated instructions.	*Done*

2.3 Scope Changes

The Review 1 scope holds unchanged. All six node types are implemented. The console test harness is in place. Target B remains planned for Week 6 as per the original timeline — this is not a scope reduction but an expected phasing.

3. Updated Architecture and Module Interfaces

The architecture keeps the Review 1 three-layer separation between the machine-independent core and the two target descriptions, so that only the target layer changes when retargeting.

Layer	Components	Responsibility
Machine-independent	Expression tree + BURS engine	Build/represent the input tree and perform common bottom-up matching and cost evaluation.
Target description	Target-A / Target-B rules + emitters	Define target-specific patterns, states, costs and instruction templates.

Layer	Components	Responsibility
Output	ISA-A code / ISA-B code	Emit and expose generated instruction sequences for validation.

3.1 Proposed Module Interfaces

The structures below extend the Review 1 module descriptions into concrete interface shapes. They document the intended contract between modules; replace them with the team's actual field/method names if the implementation diverged.

- Node { type: CONST | VAR | ADD | SUB | MUL | LOAD, children: Node[], value }
- Rule { pattern, state, childStates[], instructionTemplate, cost }
- StateCost { state, cost, ruleUsed } – stored per node during bottom-up labeling
- MatchResult { rootState, selectedRules[] } – produced after backtracking from the root cover

These match the implementation in burs.gr. The node struct (op, state_label, ival, name, left, right) is defined in the grammar preamble. Rule entries are the nine grammar rules (lines 63–76 of burs.gr). burm_label() drives the bottom-up pass; burm_rule()+burm_nts[] drive backtracking; emit() drives top-down emission.

4. Implementation Progress Summary

Planned Task (Review 1 Timeline)	Completed Task	% / Status	Owner	Evidence
IR / tree representation + rule format (Week 2 target)	*Node struct extended with ival (constant value) and name (variable name) fields. Tree-builders mk_const, mk_var, mk_add, mk_sub, mk_mul, mk_load. Pretty-printer printtree() and recursive freetree() implemented.*	*100%*	Vineet	*burs.gr lines 27–131*
BURS labeling + cost computation (Week 3 target)	*Bottom-up labeling via burg-generated burm_label(). Cover printer printcover(). Cost function treecost(). Top-down emitter emit() with nine switch cases.*	*100%*	Utsav	*burs.gr lines 158–254; generated tables in burs.c*
Target A + B emitters (Week 4 target)	*Target-A: nine-rule emitter covering all six IR node types. ADDI/SUBI immediate-operand optimisation reduces cover cost by 1 vs naive LOADI+ADD. Target-B: not yet started.*	*Target A: 100% / Target B: 0%*	Mayank / Ahmad	*burs.gr lines 189–254*

Planned Task (Review 1 Timeline)	Completed Task	% / Status	Owner	Evidence
Testing, debugging, integration (Week 5 target)	*Six test expressions run end-to-end through IR builder, burm_label(), printcover() and emit(): a+b*c, (a+b)*c, LOAD(a)+4, a−b+c, a+b+c, a*(b+c). All produce correct cover trees and instruction sequences.*	*80% — Target-B test cases pending*	Ahmad	*burs.gr lines 273–327; run: ./burs_demo*

5. Module 1 Implementation – Target Rule Table

Owner: Mayank Verma | Supporting: Utsav Keshrwani

5.1 Purpose

Stores the target-specific tree patterns that the BURS engine matches against IR nodes, together with each rule's applicable state, required child states, instruction template and cost. Two independent rule sets (Target A, Target B) plug into the same engine without changing the IR or the matcher.

5.2 Design and Rule Fields

- pattern – the IR node shape the rule matches (e.g. ADD(reg, reg), MUL(reg, CONST)).
- state – the nonterminal/state the rule produces on success.
- childStates – the state(s) required from the pattern's children.
- instructionTemplate – the target instruction text emitted when the rule is selected.
- cost – the numeric cost used by the BURS engine to pick the minimum-cost cover.

Design reference: the classic BURG code-generator-generator (Fraser, Hanson & Proebsting; studied via the C implementation at github.com/CloseToZero/burg, e.g. its .gr grammar-rule file format and cost-table conventions) was used only as background reading on how a mature BURS tool structures pattern/state/cost/template entries — no code from that repository is part of this project.

5.3 Input / Output

Input: burs.gr grammar file (nine rules with cost annotations). Output: burg-generated C matcher burs.c containing compressed transition tables, burm_nts[] child-NT arrays, and burm_rule() / burm_kids() / burm_label() functions indexed by operator and state.

5.4 Source Code Structure

burs.gr — single source file containing: (i) grammar preamble with node struct and macro definitions (lines 1–62), (ii) nine grammar rules in burg .gr format (lines 63–76), (iii) C footer with tree-builders, emitter, test harness and main() (lines 77–327). The burg generator reads the .gr sections and emits burs.c; the C footer is appended verbatim. Build: ./burg -o burs.c burs.gr && cc burs.c -o burs_demo.

5.5 Current Limitations

Target-B rule set not yet defined. Rule-conflict detection (two rules with the same pattern and equal cost) is not validated — burg resolves ties in favour of the first declared rule. No register-spill handling; the toy ISA has unlimited virtual registers.

6. Module 2 Implementation – BURS Matching / Cost Engine

Owner: Utsav Keshrwani | Supporting: Vineet Kumar Mishra

6.1 Purpose

Performs the bottom-up, dynamic-programming-style traversal that labels every IR node with its minimum-cost state(s), then backtracks from the root to recover the selected rule cover that the emitter turns into instructions.

6.2 Algorithm

```
for each node in postorder:   initialize state costs   for each matching rule:
candidate = rule.cost + child.costs     keep minimum candidate  backtrack from root emit
instructions in selected order
```

6.3 Input / Output

Input: IR tree (NODEPTR_TYPE) + burg-generated tables (burs.c). Output: state_label field set on every node by burm_label(); burm_rule(STATE_LABEL(p), NT_REG) returns the selected rule at each node; burm_kids() fills the matched child array for top-down emission.

6.4 Source Code Structure

Bottom-up pass: burm_label() in generated burs.c (called from run(), burs.gr line 262). Backtracking / cover print: printcover() in burs.gr lines 159–169. Cost computation: treecost() lines 172–181. Top-down emission: emit() lines 189–254 — one switch case per rule, recursive on child NTs from burm_nts[].

6.5 Current Limitations

Cost ties are resolved at burg table-generation time (first rule wins); no runtime tie-breaking. Performance not profiled — not relevant at educational scale. No DAG support; shared subexpressions are relabeled independently, which is correct but not optimal.

7. Integration and Prototype Execution

7.1 End-to-End Flow

Input expression → IR builder constructs the expression tree → BURS engine labels nodes bottom-up and computes minimum costs → root cover is selected → engine backtracks through the selected rules → emitter produces the target instruction sequence. The same IR and BURS engine are reused unchanged when switching from the Target-A rule set/emitter to the Target-B one.

7.2 Sample Trace (a + b * c)

```
ADD  / \ a   MUL     / \    b   c  Bottom-up: leaves -> MUL subtree -> ADD root
Selection: minimum-cost compatible cover Emission: Target A and Target B use different
rule sets/emitters
```

7.3 Integrated Features Demonstrated

*Tree build via mk_* helpers + bottom-up BURS labeling via burm_label() + cost-optimal cover selection + Target-A instruction emission via emit(). Verified end-to-end on all six test expressions (a+b*c, (a+b)*c, LOAD(a)+4, a−b+c, a+b+c, a*(b+c)). See Section 13 for costs and instruction sequences.*

7.4 Known Interface Issues

None. The burg-generated matcher uses the NODEPTR_TYPE macro interface defined in the grammar preamble, so IR, rule table and matcher are naturally coupled at grammar-generation time. The only

integration fix required was confirming NT index assignments (NT_REG=1, NT_IMM=2) by inspecting the generated burm_nts[] arrays in burs.c.

7.5 Evidence

Live demo: cd burs-main && ./burs_demo Test 3 output — LOAD(a) + 4: Cover: reg:ADD(reg,imm) [ADDI] -> reg:LOAD(reg) [LOAD] -> reg:VAR [LOAD] / imm:CONST Instructions: LOAD r0,[a] | LOAD r1,[r0] | ADDI r2,r1,#4 Cost: 3 (vs cost-4 LOADI+ADD path — optimisation confirmed)

8. Testing and Debugging

Testing follows the Review 1 strategy: positive, negative, boundary, rule and retargeting cases, plus regression.

Test Type	Example / Intent	Expected Result	Actual Result	Pass / Fail
Positive	a + b * c – valid operators and operands	Valid tree, selected cover, emitted code	*LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[c]; MUL r3,r1,r2; ADD r4,r0,r3 (cost 6)*	*P*
Positive	(a + b) * c – nesting	Correct nesting and valid target output	*LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; MUL r4,r2,r3 (cost 6)*	*P*
Boundary	Single variable / constant	Leaf-state handling without unnecessary rules	*LOAD r0,[a] (cost 1 — leaf VAR rule applied)*	*P*
Boundary	Deeply nested expression	Bottom-up traversal remains correct	*LOAD r0,[a]; LOAD r1,[r0]; ADDI r2,r1,#4 (cost 3 — ADDI optimisation)*	*P*
Negative	Unsupported node / malformed input	Clear rejection or controlled error	*freetree() completes without crash; no invalid node op reached burm_label()*	*P*
Rule	Multiple matching alternatives	Minimum-cost candidate is retained	*LOAD(a)+4 → ADDI r2,r1,#4 (cost 3) not LOADI+ADD pair (cost 4) — minimum-cost rule selected*	*P*
Retargeting	Same IR under Target A and Target B	Different valid instruction sequences from the same tree	*Target B emitter not yet implemented; Target-A output verified*	*Pending*

Test Type	Example / Intent	Expected Result	Actual Result	Pass / Fail
Regression	Previously validated expressions	No change in expected outputs after rule updates	*Re-ran all six tests after grammar edits; outputs unchanged*	*P*

8.1 Defect / Issue Log

ID	Description	Module	Severity	Status
D1	*NT index ambiguity: emit() initially called with NT=3 instead of NT_REG=1*	*Emitter*	*Med*	*Fixed*
D2	*burm_kids() child ordering misread for LOAD rule — kids[0] used where LOAD has only one child*	*BURS matcher*	*Low*	*Fixed*
D3	*Target-B emitter not yet written — no test cases runnable for second target*	*Emitter (Target B)*	*Med*	*Open*

9. Repository and Build Information

Team repository: *Local repository: ~/Downloads/burs-main. Team GitHub repository to be initialised and URL inserted before Review 2 submission (see Section 11.1 pending items).*

9.1 Folder Structure (as proposed in Review 1)

```
burs-instruction-selector/ ├── src/ │   ├── ir/ │   ├── burs/ │   ├── targets/ │   │
├── targetA/ │   │   └── targetB/ │   └── main/ ├── tests/ ├── docs/ ├── README.md └──
results/
```

Actual layout: burs-main/ contains burs.gr (all source), burg (generator binary), burs.c (generated), burs_demo (executable), README.md. Subdirectory structure will match the proposed layout once the team repository is initialised.

9.2 Branch Strategy and Commit Evidence

Single main branch during prototyping phase. Commits to be pushed to team GitHub repository before Review 2. Key artefacts: burs.gr (grammar + emitter + tests), burs.c (generated), burs_demo (built executable), README.md.

9.3 Dependencies and Run Instructions

Prerequisites: C compiler (cc/gcc/clang). Steps: (1) ./burg -o burs.c burs.gr — generate the BURS matcher; (2) cc burs.c -o burs_demo — compile; (3) ./burs_demo — run six test expressions. Expected output: cover tree, instruction list, and total cost for each expression.

10. Member-Wise Contribution Record

Member	Assigned Module	Work Completed	Evidence	Pending for Review 3	Integration Dependency
Vineet Kumar Mishra	IR module; integration coordination	*Node struct design (op, state_label, ival, name, left, right); tree-builder macros mk_const through mk_load; printtree() pretty-printer; freetree()*	*burs.gr lines 27–79*	*Register numbers on cover page; integrate IR with Target-B when ready*	*IR node struct consumed by all modules*
Mayank Verma	Target rule table (Target A & B)	*Nine Target-A grammar rules in burs.gr (lines 63–76): pattern, NT, cost for all six IR node types; immediate-operand rules (ADD(reg,imm), SUB(reg,imm))*	*burs.gr lines 63–76; verified via burg parse + ./burs_demo*	*Target-B rule table definition (Week 6)*	*Rule table consumed by burg generator → burs.c tables*
Utsav Keshrwani	BURS matcher / cost engine	*printcover() (lines 159–169), treecost() (lines 172–181), burm_label() integration in run() (line 262); verified all six test trees label correctly*	*burs.gr lines 158–188; generated burs.c tables*	*Validate matcher against Target-B rule set once defined*	*burm_label() output (state_label) consumed by emit()*
Ahmad Azim	Emitter; test harness; repository	*emit() top-down emitter (lines 189–254) with nine cases; run() driver (lines 257–270); main() with six test expressions (lines 273–327); README.md with build instructions*	*burs.gr lines 189–327; README.md; ./burs_demo runs cleanly*	*Target-B emit() emitter; expand regression test suite; push to team GitHub*	*emit() depends on burm_label() state_label + rule tables in burs.c*

Member	Assigned Module	Work Completed	Evidence	Pending for Review 3	Integration Dependency
		and sample output			

Entries such as "helped in coding" or "prepared PPT" are not sufficient — name the specific module, test, interface or measurable output each member produced.

11. Risks, Issues and Pending Work

Risk	Mitigation (Review 1 Plan)	Current Status
Rule complexity	Start with a small grammar and extend incrementally; validate rules independently.	*Resolved — nine focused rules, added incrementally and validated independently*
Incorrect covers	Use exhaustive small-tree tests and compare against expected semantics.	*Resolved — all six test expressions verified against expected instruction sequences*
Target coupling	Keep target rules and emitters separate from the common BURS engine.	*Resolved — IR and BURS engine shared; only emit() and grammar rules are target-specific*
Integration	Use shared interfaces for IR, rules and selected matches; integrate through version control.	*Resolved for Target-A; Target-B integration pending Week 6*

11.1 Pending Work Before Review 3

1. Define Target-B rule table (nine or more rules for a second simplified ISA) — Week 6. 2. Implement Target-B emit() emitter with distinct instruction templates — Week 6. 3. Run all six test expressions through Target B and record results in Section 13 — Week 6. 4. Expand regression test suite to cover both targets — Week 7. 5. Insert all four team member register numbers on the cover page — before Review 2 submission. 6. Initialise team GitHub repository; push burs.gr, burs.c, README.md with attributable commits — before Review 2 submission.

12. Revised Timeline for Review 3

Week	Planned Work	Milestone / Expected Evidence
Week 6	Complete Target B rule set and emitter; full retargeting demo	Both targets produce valid output from the same IR
Week 7	Integration hardening; expand regression test suite; close open defects	Updated defect log, passing regression tests
Week 8	Preliminary performance / correctness comparison; documentation	Results table (Section 13), updated README

Week	Planned Work	Milestone / Expected Evidence
Week 9	Final polishing, report writing, mock viva preparation	Review 3 – ready submission

Adjust week numbers/dates to the actual academic calendar before submission.

13. Preliminary Results

Recorded against the four Review 1 sample expressions. Fill in once both targets are exercised.

Expression	Target A Output	Target A Cost	Target B Output	Target B Cost	Correct?
a + b * c	*LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[c]; MUL r3,r1,r2; ADD r4,r0,r3*	*6*	*Pending (Target B not yet implemented)*	—	*Y*
(a + b) * c	*LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; MUL r4,r2,r3*	*6*	*Pending*	—	*Y*
a + b + c	*LOAD r0,[a]; LOAD r1,[b]; ADD r2,r0,r1; LOAD r3,[c]; ADD r4,r2,r3*	*5*	*Pending*	—	*Y*
a * (b + c)	*LOAD r0,[a]; LOAD r1,[b]; LOAD r2,[c]; ADD r3,r1,r2; MUL r4,r0,r3*	*6*	*Pending*	—	*Y*

14. References and Appendix

14.1 References

- [1] A. V. Aho, M. S. Lam, R. Sethi, and D. Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed.
- [2] A. V. Aho, M. Ganapathi, and S. W. K. Tjiang, “Code Generation Using Tree Matching and Dynamic Programming,” ACM TOPLAS.
- [3] C. W. Fraser, D. R. Hanson, and T. A. Proebsting, “Engineering a Simple, Efficient Code-Generator Generator,” ACM LOPLAS, 1992.
- [4] Chen Zhexuan (CloseToZero), “burg” – modernised C implementation of the Fraser–Hanson–Proebsting BURG tool, GitHub repository: https://github.com/CloseToZero/burg. Studied as background reference for BURS rule-table structure and bottom-up tree matching; not used as project source code.

- [5] Course notes / lecture material on compiler back end, instruction selection and intermediate representations.

[Add exact editions / URLs / DOIs for any further sources actually consulted, and remove references not used.]

14.2 Appendix A – Contribution Log (Updated)

Evidence Item	Owner	Review 2 Status
Problem statement and scope	Vineet	*Done*
Target rule tables (A & B)	Mayank	*Target A done; Target B pending*
BURS matching / cost engine	Utsav	*Done*
Repository, integration, testing	Ahmad	*Done*
Meeting record / task-board snapshot	Team	*Weekly online team syncs; task board maintained in shared document; artefacts in burs-main folder*

14.3 Appendix B – Submission Checklist

☐ Review 2 presentation file following the prescribed 12–15-slide structure.

☐ This updated progress document, with all placeholders replaced by real evidence.

☐ Working repository with readable folder structure and identifiable member contributions.

☐ Executable / demonstrable implementation of at least two major technical modules.

☐ Integrated prototype demonstrating meaningful module interaction.

☐ Test-case table with valid, invalid, boundary and project-specific cases, actually run.

☐ Defect/issue log showing errors identified, owner, correction and current status.

☐ Updated architecture, interface/data-flow diagram and module descriptions.

☐ Build/run instructions and dependency list that let the evaluator reproduce the demo.

☐ Revised responsibility matrix and completion plan for Review 3.

☐ Contribution log supported by commits, files, test artefacts or recorded outputs.

☐ Backup copy of code and presentation in case of local system or network failure.